

ESPM — Data Integration — 2026

Pipeline de Indicadores Macroeconômicos Brasileiros

Relatório Técnico — Projeto Final

AUTORES

Andre Henrique Pacheco Alves

Enrico Comassetto Di Gioia

Enzo de Moraes Godoy

Maria Luiza Oliveira de Souza

Luzivania Jesus Bonfim

1. Sumário Executivo

Este documento descreve o projeto Pipeline de Indicadores Macroeconômicos Brasileiros, desenvolvido como trabalho final da disciplina Data Integration (ESPM 2026.1). O objetivo central é construir um pipeline ETL completo — da extração à carga — que consolide dados heterogêneos do Banco Central do Brasil (BCB) e do Conselho Monetário Nacional (CMN) num Data Warehouse relacional, orquestrado pelo Apache Airflow em ambiente Docker.

O sistema extrai automaticamente três indicadores macroeconômicos — taxa SELIC, IPCA mensal e câmbio USD/BRL — da API pública do BCB (formato JSON), além de uma série histórica de metas de inflação via arquivo CSV estático. Após validações de qualidade, os dados são normalizados para um modelo dimensional (star schema) e carregados via upsert idempotente no Postgres, com agendamento semanal via Airflow.

Indicador	Valor
Séries extraídas	3 (API BCB) + 1 CSV
Cobertura histórica	24 meses (configurável)
Registros de fato	~2.500 (diário + mensal)
Tabelas no DW	4 (3 dimensões + 1 fato)
Tasks na DAG	5 (init_db → extract → validate → transform → load)
Testes automatizados	24 casos (pytest)
Tecnologias	Python 3.11, Airflow 2.9, PostgreSQL 15, Docker

Tabela 1 — Indicadores quantitativos do projeto

2. Objetivos

O projeto tem como objetivo principal demonstrar a implementação de um pipeline de integração de dados de ponta a ponta, abrangendo todas as etapas do processo ETL com boas práticas de engenharia de dados. Os objetivos específicos são:

- Extrair dados de fontes heterogêneas (REST API JSON e arquivo CSV) de forma automatizada
- Aplicar validações de qualidade nos dados brutos antes da transformação
- Normalizar os dados para um modelo dimensional (star schema) adequado para análises
- Carregar os dados via upsert idempotente, permitindo re-execuções sem duplicatas
- Orquestrar todo o fluxo com Apache Airflow, incluindo agendamento e retries automáticos
- Garantir reprodutibilidade total via Docker Compose com um único comando de inicialização
- Cobrir a lógica ETL com testes automatizados (pytest)

3. Arquitetura da Solução

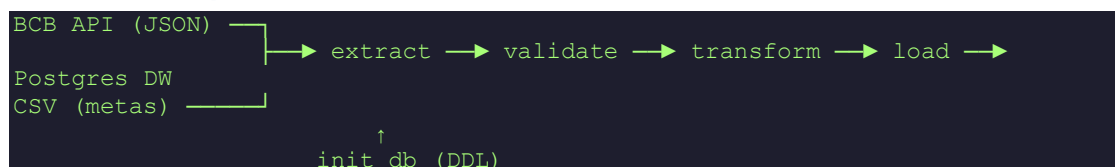
3.1 Visão Geral do Fluxo

O pipeline segue a arquitetura ETL dividida em cinco tasks sequenciais, orquestradas pela DAG macro_pipeline no Apache Airflow. Os dados fluem das fontes externas até o Data Warehouse em Postgres, passando por etapas de validação e transformação intermediárias:

Task	Módulo	Responsabilidade	Saída
init_db	SQLAlchemy	Cria schema dw e tabelas (IF NOT EXISTS)	Schema Postgres
extract	extract.py	GET API BCB + leitura CSV	raw_*.json, raw_metas.parquet
validate	validate.py	Nulos, ranges, duplicatas pré-transform	Exceção ou OK
transform	transform.py	Normaliza para modelo dimensional	dim_*.parquet, fact_*.parquet
load	load.py	Upsert nas tabelas do DW	DW atualizado

Tabela 2 — Tasks da DAG e suas responsabilidades

3.2 Fluxo do Pipeline



3.3 Decisões de Arquitetura

Arquivos intermediários em vez de XCom: com 24 meses de cotações diárias (~1.500 registros × 3 séries), o payload excede o limite padrão de 48 KB do XCom do Airflow. Gravar em data/tmp/ (volume Docker compartilhado) resolve o problema e mantém os dados brutos acessíveis para debug.

Upsert idempotente (ON CONFLICT DO UPDATE): toda inserção usa INSERT ... ON CONFLICT, tornando o pipeline idempotente. Re-execuções e correções não geram duplicatas. Essencial no Airflow, onde retries são comuns.

DDL em arquivo SQL externo: o arquivo create_tables.sql é executado a cada run via init_db. Isso mantém separação clara entre infraestrutura (SQL) e lógica de negócio (Python), além de permitir versionamento independente do schema.

LocalExecutor em vez de CeleryExecutor: suficiente para o volume e número de workers desta carga. Reduz drasticamente a complexidade do docker-compose, eliminando Redis e workers extras.

PostgreSQL único com dois bancos: os bancos airflow (metadados) e dw (Data Warehouse) convivem no mesmo container. Em produção, seriam instâncias separadas com SLAs e backups distintos.

4. Fontes de Dados

4.1 API do Banco Central do Brasil

A API pública do BCB (api.bcb.gov.br) fornece dados históricos de séries temporais econômicas. É acessada via HTTP GET sem autenticação, retornando JSON semi-estruturado no formato [{"data": "DD/MM/YYYY", "valor": "X.XX"}].

Chave Interna	Código BCB	Nome	Periodicidade	Unidade
selic_diaria	432	SELIC — Taxa Over	Diária	% a.a.
ipca_mensal	433	IPCA — Variação Mensal	Mensal	%
cambio_usd	1	Câmbio USD/BRL (venda)	Diária	R\$

Tabela 3 — Séries extraídas da API BCB

4.2 CSV de Metas de Inflação (CMN)

Arquivo CSV estático com a série histórica de metas de inflação definidas pelo Conselho Monetário Nacional. Cobre o período 2015–2025 e inclui meta central, bandas superior/inferior e inflação efetivamente realizada (IPCA acumulado do ano).

Coluna	Tipo	Descrição
ano	Integer	Ano-referência da meta
meta_inflacao	Float	Meta central do CMN (%)
banda_superior	Float	Limite superior da banda (%)
banda_inferior	Float	Limite inferior da banda (%)
inflacao_realizada	Float	IPCA acumulado realizado (%)

Tabela 4 — Estrutura do CSV de metas de inflação

5. Modelo Dimensional (Star Schema)

O DW adota um modelo dimensional (star schema) com uma tabela fato central e três dimensões. O workload é analítico — agregações temporais e cruzamentos entre séries — o que torna o star schema mais eficiente que um esquema relacional normalizado, reduzindo o número de JOINS nas queries de valor.

5.1 Tabela Fato

Coluna	Tipo	Descrição
id	SERIAL PK	Surrogate key gerada automaticamente
data	DATE FK	Referência para dim_tempo
indicador_id	INTEGER FK	Referência para dim_indicador
valor	NUMERIC(12,6)	Valor do indicador na data

Tabela 5 — fact_serie_temporal (grão: 1 valor por indicador por data)

5.2 Dimensões

Dimensão	PK	Cardinalidade	Notas
dim_tempo	data	~730 linhas	Atributos: dia, mês, ano, trimestre, semestre, nome_mês
dim_indicador	id	3 linhas	Metadados estáticos das séries (código BCB, unidade, etc.)
dim_meta_inflacao	ano	11 linhas	Dimensão degenerada — JOIN pelo ano, não por FK do fato

Tabela 6 — Dimensões do DW

6. Validações de Qualidade de Dados

O módulo `validate.py` implementa quatro regras de qualidade aplicadas antes da transformação (sobre dados brutos) e durante a transformação (integridade referencial):

#	Regra	Colunas alvo	Comportamento ao violar
1	Nulos em campos críticos	data, valor	Levanta <code>ValidationError</code> — aborta pipeline
2	Duplicatas na chave	(data, indicador_id)	Loga aviso, remove (keep=first)
3	Range de valores	valor por série	Loga aviso, remove outliers do fato
4	Integridade referencial	fact.data → dim_tempo.data	Levanta <code>ValidationError</code> — aborta pipeline

Tabela 7 — Regras de validação implementadas

Série	Mínimo	Máximo	Justificativa
selic_diaria	0,0%	50,0%	SELIC nunca foi negativa; 50% cobre choques históricos extremos
ipca_mensal	-5,0%	30,0%	Deflação pontual é possível; 30% cobre hiperinflação residual
cambio_usd	R\$ 0,50	R\$ 20,00	Margem ampla para cenários de crise cambial severa

Tabela 8 — Ranges de validação por série

7. Detalhamento das Etapas ETL

7.1 Extração (`extract.py`)

A função `extract_bcb()` itera sobre o dicionário `SERIES_CONFIG` e realiza chamadas HTTP GET à API do BCB com parâmetros de data (janela de 24 meses configurável via variável de ambiente `BCB_MONTHS_HISTORY`). Cada série é persistida individualmente como JSON em `data/tmp/raw_{chave}.json`.

A função `extract_csv()` lê o arquivo `metas_inflacao.csv` via `pandas` e o converte para `Parquet` (`raw_metas.parquet`), preservando tipos sem necessidade de re-parsear strings em etapas posteriores.

7.2 Validação (`validate.py`)

Executada como `task` independente antes da transformação, a validação opera sobre os arquivos brutos (`.json` e `.parquet`). Isso garante que problemas de qualidade sejam detectados o mais cedo possível no pipeline, evitando processamento desnecessário e dados corrompidos no DW.

7.3 Transformação (`transform.py`)

A função `transform_all()` orquestra a construção de todas as entidades dimensionais:

- `dim_indicador`: construída estaticamente a partir do dicionário `SERIES_CONFIG` — não depende de dados externos
- Fatos brutos: cada registro BCB é parseado com `_parse_bcb_record()`, convertendo datas `DD/MM/YYYY` e valores `string` para `float`
- `dim_tempo`: gerada dinamicamente a partir do conjunto de datas únicas presentes nos fatos
- Integridade referencial: verificada antes da persistência (toda data do fato deve existir na `dim_tempo`)
- `dim_meta_inflacao`: lida do `Parquet`, enriquecida com a coluna derivada `bateu_meta` (booleano)

7.4 Carga (`load.py`)

Os quatro loaders seguem a mesma estratégia: leitura do `Parquet` correspondente, iteração linha a linha e execução de `INSERT ... ON CONFLICT DO UPDATE` via `SQLAlchemy`. A ordem de carga respeita as constraints de FK: dimensões primeiro, fato por último.

```
-- Upsert do fato (estratégia idempotente)
INSERT INTO dw.fact_serie_temporal (data, indicador_id, valor)
VALUES (:data, :indicador_id, :valor)
ON CONFLICT (data, indicador_id) DO UPDATE
SET valor = EXCLUDED.valor
```

8. Queries Analíticas

Três queries SQL foram desenvolvidas para demonstrar o valor analítico do DW, cada uma respondendo a uma pergunta de negócio relevante.

Query 1 — Evolução da SELIC (média mensal)

Responde: "Como a taxa básica de juros variou mês a mês nos últimos 24 meses?"

```
SELECT t.ano, t.mes, t.nome_mes,
       ROUND(AVG(f.valor)::numeric, 4) AS selic_media_mensal_pct
FROM dw.fact_serie_temporal f
JOIN dw.dim_tempo t ON f.data = t.data
JOIN dw.dim_indicador i ON f.indicador_id = i.id
WHERE i.chave = 'selic_diaria'
GROUP BY t.ano, t.mes, t.nome_mes
ORDER BY t.ano, t.mes;
```

Resultado — 26 linhas retornadas, cobrindo maio/2024 a junho/2026:

```

Colunas: ['ano', 'mes', 'nome_mes', 'selic_media_mensal_pct']
Linhas retornadas: 26

(2024, 5, 'May', Decimal('10.5000'))
(2024, 6, 'June', Decimal('10.5000'))
(2024, 7, 'July', Decimal('10.5000'))
(2024, 8, 'August', Decimal('10.5000'))
(2024, 9, 'September', Decimal('10.6000'))
(2024, 10, 'October', Decimal('10.7500'))
(2024, 11, 'November', Decimal('11.1500'))
(2024, 12, 'December', Decimal('11.8952'))
(2025, 1, 'January', Decimal('12.3145'))
(2025, 2, 'February', Decimal('13.2500'))
(2025, 3, 'March', Decimal('13.6371'))
(2025, 4, 'April', Decimal('14.2500'))
(2025, 5, 'May', Decimal('14.6371'))
(2025, 6, 'June', Decimal('14.8500'))
(2025, 7, 'July', Decimal('15.0000'))
(2025, 8, 'August', Decimal('15.0000'))
(2025, 9, 'September', Decimal('15.0000'))
(2025, 10, 'October', Decimal('15.0000'))
(2025, 11, 'November', Decimal('15.0000'))
(2025, 12, 'December', Decimal('15.0000'))
(2026, 1, 'January', Decimal('15.0000'))
(2026, 2, 'February', Decimal('15.0000'))
(2026, 3, 'March', Decimal('14.8952'))
(2026, 4, 'April', Decimal('14.7417'))
(2026, 5, 'May', Decimal('14.5000'))
(2026, 6, 'June', Decimal('14.5000'))

```

Figura 1 — Saída da Query 1: evolução da SELIC média mensal (10,50% → 14,50%)

Query 2 — IPCA Acumulado × Meta CMN

Responde: "O Brasil cumpriu a meta de inflação em cada ano?"

```

SELECT t.ano,
       ROUND(SUM(f.valor)::numeric, 2) AS ipca_acumulado_pct,
       m.meta_inflacao, m.banda_inferior, m.banda_superior, m.bateu_meta
FROM dw.fact_serie_temporal f
JOIN dw.dim_tempo           t ON f.data = t.data
JOIN dw.dim_indicador       i ON f.indicador_id = i.id
LEFT JOIN dw.dim_meta_inflacao m ON t.ano = m.ano
WHERE i.chave = 'ipca_mensal'
GROUP BY t.ano, m.meta_inflacao, m.banda_inferior, m.banda_superior,
m.bateu_meta
ORDER BY t.ano;

```

Resultado — 3 anos retornados (2024, 2025, 2026):

```

=====
Executando: IPCA Acumulado por Ano
=====
Colunas: ['ano', 'ipca_acumulado_pct', 'meta_inflacao', 'banda_inferior', 'banda_superior', 'bateu_meta']
Linhas retornadas: 3

(2024, Decimal('2.94'), Decimal('3.00'), Decimal('1.50'), Decimal('4.50'), False)
(2025, Decimal('4.19'), Decimal('3.00'), Decimal('1.50'), Decimal('4.50'), False)
(2026, Decimal('2.58'), None, None, None, None)

```

Figura 2 — Saída da Query 2: IPCA acumulado por ano vs. meta CMN (bateu_meta=False nos dois anos completos)

Query 3 — Câmbio USD/BRL por Trimestre

Responde: "Qual foi a volatilidade cambial em cada trimestre?"

```
SELECT t.ano, t.trimestre,
       ROUND(AVG(f.valor)::numeric, 4) AS cambio_medio,
       ROUND(MAX(f.valor)::numeric, 4) AS cambio_max,
       ROUND(MIN(f.valor)::numeric, 4) AS cambio_min,
       ROUND((MAX(f.valor) - MIN(f.valor))::numeric, 4) AS amplitude
FROM dw.fact_serie_temporal f
JOIN dw.dim_tempo t ON f.data = t.data
JOIN dw.dim_indicador i ON f.indicador_id = i.id
WHERE i.chave = 'cambio_usd'
GROUP BY t.ano, t.trimestre
ORDER BY t.ano, t.trimestre;
```

Resultado — 9 trimestres retornados (Q2/2024 a Q2/2026):

```
=====
Executando: Câmbio por Trimestre
=====
Colunas: ['ano', 'trimestre', 'cambio_medio', 'cambio_max', 'cambio_min', 'amplitude']
Linhas retornadas: 9

(2024, 2, Decimal('5.3329'), Decimal('5.5589'), Decimal('5.1443'), Decimal('0.4146'))
(2024, 3, Decimal('5.5454'), Decimal('5.7646'), Decimal('5.3967'), Decimal('0.3679'))
(2024, 4, Decimal('5.8369'), Decimal('6.1991'), Decimal('5.4305'), Decimal('0.7686'))
(2025, 1, Decimal('5.8522'), Decimal('6.2086'), Decimal('5.6628'), Decimal('0.5458'))
(2025, 2, Decimal('5.6661'), Decimal('6.0605'), Decimal('5.4571'), Decimal('0.6034'))
(2025, 3, Decimal('5.4488'), Decimal('5.6034'), Decimal('5.3010'), Decimal('0.3024'))
(2025, 4, Decimal('5.3955'), Decimal('5.5739'), Decimal('5.2729'), Decimal('0.3010'))
(2026, 1, Decimal('5.2591'), Decimal('5.4372'), Decimal('5.1382'), Decimal('0.2990'))
(2026, 2, Decimal('5.0226'), Decimal('5.1763'), Decimal('4.8973'), Decimal('0.2790'))
```

Figura 3 — Saída da Query 3: câmbio USD/BRL por trimestre com média, máxima, mínima e amplitude

9. Testes Automatizados

O projeto cobre a lógica ETL com 24 casos de teste organizados em 9 classes, implementados com pytest. Os testes focam nas funções de transformação e validação, que concentram as regras de negócio críticas:

Classe de Teste	Função testada	Casos	Cobertura principal
TestParseBcbRecord	<code>_parse_bcb_record()</code>	6	Parsing de datas, valores, campos ausentes/inválidos
TestBuildDimTempo	<code>build_dim_tempo()</code>	4	Colunas, semestre, trimestre, deduplicação
TestBuildDimIndicador	<code>build_dim_indicador()</code>	2	Cardinalidade e chaves esperadas
TestTransformMetas	<code>_transform_metas()</code>	3	bateu_meta, colunas de saída, valores nulos
TestCheckNulls	<code>check_nulls()</code>	2	Levanta exceção e cenário OK
TestCheckDuplicates	<code>check_duplicates()</code>	2	Remoção e cenário sem duplicatas

Classe de Teste	Função testada	Casos	Cobertura principal
TestCheckRange	check_range()	2	Remoção de outliers e modo strict
TestCheckReferentialIntegrity	check_referential_integrity()	2	Cenário OK e detecção de órfãos
Total	—	24	—

Tabela 9 — Suite de testes automatizados

Como executar os testes:

```
# Instalar dependências
pip install -r requirements.txt

# Rodar todos os testes
pytest tests/ -v

# Com relatório de cobertura
pytest tests/ -v --tb=short
```

10. Guia de Execução

10.1 Pré-requisitos

- Docker 24+
- Docker Compose v2
- Git

10.2 Inicialização

```
# 1. Clonar o repositório
git clone <URL_DO_REPO> && cd <pasta>

# 2. Configurar variáveis de ambiente
cp .env.example .env

# 3. Subir todos os serviços
docker compose up --build -d

# 4. Aguardar inicialização (~60s) e acessar o Airflow
# http://localhost:8080 (usuário: admin / senha: admin)
```

10.3 Execução da DAG

Após o login no Airflow Webserver (localhost:8080), localize a DAG macro_pipeline, ative-a pelo toggle e dispare manualmente via Trigger DAG. O pipeline completo leva aproximadamente 2–3 minutos dependendo da latência da API BCB.

10.4 Verificação dos Dados

```
# Conectar ao banco dw
docker exec -it <container_postgres> psql -U dw -d dw

-- Contar registros no fato
SELECT COUNT(*) FROM dw.fact_serie_temporal;

-- Verificar dimensões
```

```
SELECT * FROM dw.dim_indicador;

-- Executar queries analíticas
\i /sql/queries.sql
```

10.5 Variáveis de Ambiente

Variável	Padrão	Descrição
BCB_MONTHS_HISTORY	24	Janela histórica em meses para extração BCB
METAS_CSV_PATH	/opt/airflow/data/	Caminho do CSV de metas dentro do container
DW_CONN_STR	postgresql+...	Connection string do Data Warehouse
AIRFLOW_WWW_USER_*	admin/admin	Credenciais do Airflow Webserver
AIRFLOW__CORE__FERNET_KEY	(gerado)	Chave de criptografia de conexões Airflow

Tabela 10 — Variáveis de ambiente configuráveis

11. Stack Tecnológica

Camada	Tecnologia	Versão	Função
Linguagem	Python 3.11	3.11	Implementação de todos os módulos ETL
Orquestração	Apache Airflow	2.9	DAG, agendamento, retries, monitoramento
Banco de Dados	PostgreSQL	15	Armazenamento do DW e metadados Airflow
Container	Docker + Compose	24 / v2	Ambiente reproduzível, isolamento de serviços
Data Wrangling	pandas	2.2.2	Transformação e manipulação de DataFrames
Serialização	pyarrow	16.1.0	Leitura/escrita de Parquet (artefatos intermediários)
HTTP Client	requests	2.32.3	Chamadas à API REST do BCB
DB Driver	psycopg2-binary	2.9.9	Adapter Postgres para SQLAlchemy
ORM/Engine	SQLAlchemy	(Airflow)	Execução de SQL e upserts no DW
Testes	pytest	8.2.2	Suite de testes automatizados

Tabela 11 — Stack tecnológica completa

12. Limitações e Possíveis Evoluções

12.1 Limitações Atuais

- Carga full-refresh: a cada execução todas as datas do período são reprocessadas, mesmo que já existam no DW. O upsert garante idempotência, mas o volume de operações crescerá com o tempo.

- Instância Postgres única: os bancos airflow e dw compartilham o mesmo container. Em produção, isso representa risco de contenção de recursos e ausência de SLAs independentes.
- Ausência de camada de observabilidade: não há alertas configurados para falhas de pipeline fora do próprio Airflow (ex: e-mail, Slack, PagerDuty).
- API BCB sem controle de rate-limiting: o código não implementa backoff exponencial para erros 429 ou timeouts prolongados.
- Dados de câmbio sem tratamento de fuso horário: os registros diários da API BCB não incluem hora da cotação, o que pode gerar ambiguidade para análises intradiárias.
- dim_meta_inflacao sem FK direta no fato: a dimensão é referenciada analiticamente por ano (JOIN), não por chave estrangeira, o que reduz a garantia de integridade em nível de banco.

12.2 Possíveis Evoluções

- Carga incremental: implementar watermark por data de última execução e extrair apenas o delta, reduzindo drasticamente o volume de requisições à API BCB.
- Dashboard interativo: adicionar Metabase ou Streamlit consumindo o Postgres para visualização das três queries analíticas em tempo real.
- CI/CD com GitHub Actions: configurar workflow que execute pytest e lint (flake8/ruff) a cada push, garantindo qualidade contínua do código.
- Expansão de séries: o dicionário SERIES_CONFIG é facilmente extensível para incluir novos indicadores do BCB (ex: IGP-M, taxa de desemprego, resultado primário).
- Separação de ambientes: utilizar instâncias Postgres distintas para metadados do Airflow e para o DW, com backups e políticas de retenção independentes.
- Tratamento de outliers configurável: expor os ranges de validação como variáveis de ambiente para facilitar ajustes sem alterar o código.

13. Uso de Inteligência Artificial

O desenvolvimento deste projeto contou com o suporte de Claude (Anthropic) nas seguintes frentes:

- Geração e revisão da arquitetura do pipeline e do modelo dimensional
- Scaffolding inicial dos módulos ETL (extract, transform, validate, load)
- Escrita do DDL (create_tables.sql) e das queries analíticas
- Estruturação e redação da documentação técnica (README, ARCHITECTURE.md)
- Revisão de boas práticas de engenharia de dados (idempotência, upsert, qualidade)

Todo o código foi revisado, testado e adaptado pelo grupo. O uso de IA foi tratado como ferramenta de produtividade e revisão, não como substituto do entendimento técnico.

14. Referências Bibliográficas

BANCO CENTRAL DO BRASIL. Sistema Gerenciador de Séries Temporais (SGS). Disponível em: <https://api.bcb.gov.br>. Acesso em: jun. 2026.

CONSELHO MONETÁRIO NACIONAL. Metas para a Inflação. Disponível em: <https://www.bcb.gov.br/controleinflacao/metainflacao>. Acesso em: jun. 2026.

APACHE AIRFLOW. Apache Airflow Documentation. Versão 2.9. Disponível em: <https://airflow.apache.org/docs/>. Acesso em: jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 15 Documentation. Disponível em: <https://www.postgresql.org/docs/15/>. Acesso em: jun. 2026.

DOCKER INC. Docker Compose Documentation. Disponível em: <https://docs.docker.com/compose/>. Acesso em: jun. 2026.

PANDAS DEVELOPMENT TEAM. pandas Documentation. Versão 2.2. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: jun. 2026.

KIMBALL, Ralph; ROSS, Margy. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. 3. ed. Indianapolis: Wiley, 2013.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.